

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CNTT

BỘ MÔN CÁC HỆ THỐNG PHÂN TÁN



**BÁO CÁO BÀI TẬP NHÓM: PHÁT TRIỂN HỆ
THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC**

Giảng viên: Kim Ngọc Bách

Nhóm: 01

Lớp: 03

Thành viên: Phạm Nhật Minh – B22DCCN543

Thăng Lê Dương – B22DCCN172

Hà Nội – 2026

MỤC LỤC

1. TÓM TẮT	5
2. GIỚI THIỆU VÀ MỤC TIÊU HỆ THỐNG.....	5
2.1. Bối cảnh	5
2.2. Mục tiêu	6
3. PHÂN TÍCH YÊU CẦU VÀ MỨC ĐỘ ĐÁP ỨNG.....	6
3.1. Yêu cầu chức năng.....	6
3.2. Yêu cầu kỹ thuật hệ thống phân tán.....	6
4. CÔNG NGHỆ SỬ DỤNG	7
4.1. Frontend	7
4.2. Backend.....	7
4.3. Lý do chọn Socket.IO	8
5. KIẾN TRÚC HỆ THỐNG	8
5.1. Mô hình tổng thể	8
5.2. Vai trò các thành phần	9
5.3. Luồng hoạt động chính	10
5.4. Vì sao hệ thống là hệ thống phân tán.....	10
6. THIẾT KẾ DỮ LIỆU VÀ LƯU TRỮ	11
6.1. User Collection	11
6.2. Document Collection	11
6.3. Trạng thái tài liệu trong memory	12
6.4. Lưu tài liệu	12
7. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU	13
7.1. Tổng quan Operational Transformation	13
7.2. Revision system	14
7.3. Các quy tắc transform cơ bản	14
7.4. Ví dụ xử lý xung đột	14
7.5. Consistency model	15
8. GIAO THỨC TRAO ĐỔI DỮ LIỆU	15

8.1. HTTP REST API	15
8.2. WebSocket/Socket.IO Events	17
8.3. Luồng reconnect.....	18
9. QUẢN LÝ NGƯỜI DÙNG ĐANG CHỈNH SỬA.....	18
9.1. Presence và typing indicator	18
9.2. Khả năng mở rộng con trỏ cộng tác.....	19
10. XỬ LÝ ĐỒNG THỜI, XUNG ĐỘT VÀ LỖI.....	19
10.1. Xử lý nhiều client đồng thời	19
10.2. Xử lý client gửi revision cũ.....	19
10.3. Xử lý lỗi thường gặp	19
11. BẢO MẬT HỆ THỐNG	20
11.1. Authentication và Authorization.....	20
11.2. Bảo vệ mật khẩu.....	20
11.3. CORS và cấu hình môi trường.....	20
11.4. Input validation và chống lạm dụng	21
12. THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	21
12.1. Mục tiêu kiểm thử	21
12.2. Kiểm thử API bằng cURL/Postman	21
12.3. Kiểm thử realtime	22
12.4. Kết quả đánh giá hiệu năng mẫu.....	22
12.5. Đánh giá kết quả	23
13. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	23
13.1. Hạn chế hiện tại.....	23
13.2. Hướng phát triển	23
14. KẾT LUẬN	24
TÀI LIỆU THAM KHẢO.....	26

1. TÓM TẮT

Đồ án xây dựng một hệ thống chỉnh sửa tài liệu văn bản cộng tác theo thời gian thực, cho phép nhiều người dùng cùng mở, chỉnh sửa và theo dõi thay đổi trên cùng một tài liệu thông qua mạng. Hệ thống sử dụng mô hình client-server tập trung, trong đó trình duyệt người dùng là client, server NestJS đóng vai trò điều phối và MongoDB lưu trữ dữ liệu bền vững.

Điểm trọng tâm của đồ án là cơ chế đồng bộ dữ liệu phân tán. Khi nhiều người dùng chỉnh sửa đồng thời, mỗi thao tác chèn/xóa được biểu diễn thành một operation, gửi về server qua Socket.IO/WebSocket. Server dùng cơ chế Operational Transformation (OT) kết hợp revision để biến đổi vị trí thao tác, giải quyết xung đột và đảm bảo các client cuối cùng hội tụ về cùng một trạng thái tài liệu.

Báo cáo trình bày đầy đủ kiến trúc hệ thống, mô hình đồng bộ dữ liệu, giao thức trao đổi dữ liệu, thiết kế lưu trữ, cơ chế xử lý đồng thời, kiểm thử và đánh giá. Các nội dung được xây dựng dựa trên README của sản phẩm đã hoàn thành.

Thành phần	Công nghệ/giải pháp
Frontend	ReactJS, TailwindCSS, Axios, Socket.IO Client
Backend	NestJS, TypeScript, Socket.IO, WebSocket Gateway
Database	MongoDB, Mongoose
Authentication	JWT, bcryptjs
Realtime sync	WebSocket/Socket.IO, Operational Transformation, revision

2. GIỚI THIỆU VÀ MỤC TIÊU HỆ THỐNG

2.1. Bối cảnh

Trong các môi trường học tập và làm việc nhóm, nhu cầu nhiều người cùng chỉnh sửa một tài liệu đang ngày càng phổ biến. Nếu mỗi người chỉnh sửa một bản riêng rồi ghép lại thủ công, hệ thống dễ gặp lỗi mất dữ liệu, ghi đè nội dung hoặc không biết phiên bản nào là mới nhất. Vì vậy, một hệ thống chỉnh sửa tài liệu cộng tác cần đảm bảo người dùng có thể thao tác đồng thời nhưng tài liệu vẫn nhất quán.

Đồ án này mô phỏng các nguyên lý quan trọng của những hệ thống như Google Docs ở phạm vi đơn giản hơn: tài liệu văn bản thuần, giao tiếp realtime qua WebSocket, server điều phối trạng thái, và cơ chế OT để xử lý xung đột.

2.2. Mục tiêu

- Cho phép người dùng đăng ký, đăng nhập, tạo, mở, xem và chỉnh sửa tài liệu văn bản.
- Hỗ trợ nhiều người dùng cùng chỉnh sửa một tài liệu theo thời gian thực.
- Đồng bộ thay đổi gần realtime giữa các client thông qua WebSocket/Socket.IO.
- Xử lý xung đột chỉnh sửa đồng thời bằng Operational Transformation.
- Lưu tài liệu trên server/database để đảm bảo dữ liệu không mất sau khi thoát trình duyệt.
- Thể hiện rõ đặc trưng hệ thống phân tán: nhiều node client, giao tiếp mạng, xử lý đồng thời, lỗi kết nối cục bộ và đồng bộ trạng thái.

3. PHÂN TÍCH YÊU CẦU VÀ MỨC ĐỘ ĐÁP ỨNG

3.1. Yêu cầu chức năng

Yêu cầu của đề bài	Thiết kế/triển khai trong hệ thống
Tạo và quản lý tài liệu	Có API tạo, lấy danh sách, xem chi tiết, cập nhật và xóa tài liệu. Frontend cung cấp giao diện tạo/mở/chỉnh sửa tài liệu.
Chỉnh sửa cộng tác realtime	Client tham gia room theo documentId, gửi operation qua Socket.IO và nhận remote-operation từ server.
Đồng bộ thay đổi	Server quản lý revision, operation history và dùng OT để biến đổi thao tác bị trễ.
Quản lý người dùng đang chỉnh sửa	Socket.IO room theo tài liệu, active-users, user-joined/user-left và typing_indicator.
Hiển thị con trỏ/vùng chọn	README ghi nhận đây là hướng phát triển tương lai; hiện có thể mở rộng thêm event cursor-move.
Lưu trữ tài liệu	MongoDB lưu User và Document; server auto-save sau khi người dùng ngừng chỉnh sửa.

3.2. Yêu cầu kỹ thuật hệ thống phân tán

Yêu cầu kỹ thuật	Cách hệ thống thể hiện
------------------	------------------------

Yêu cầu kỹ thuật	Cách hệ thống thể hiện
Giao tiếp mạng client-server	HTTP REST API cho xác thực/CRUD tài liệu; WebSocket/Socket.IO cho realtime editing.
Xử lý nhiều client đồng thời	Mỗi browser là một client độc lập; nhiều socket có thể cùng tham gia một document room.
Cơ chế đồng bộ thay đổi	Operation-based synchronization, revision counter, OT transform và broadcast.
Xử lý xung đột	Khi client gửi operation dựa trên revision cũ, server transform operation qua lịch sử các operation đã áp dụng.
Quản lý trạng thái phân tán	Client giữ local state, server giữ authoritative state trong memory, MongoDB giữ trạng thái bền vững.

4. CÔNG NGHỆ SỬ DỤNG

4.1. Frontend

Công nghệ	Vai trò
ReactJS	Xây dựng giao diện người dùng dạng component, quản lý trạng thái editor.
React Hooks	Quản lý state, side effect, lifecycle kết nối socket.
TailwindCSS	Thiết kế giao diện nhanh, nhất quán.
Axios	Gửi HTTP request đến backend cho login/register/document CRUD.
Socket.IO Client	Kết nối realtime, emit/listen các event chỉnh sửa tài liệu.

4.2. Backend

Công nghệ	Vai trò
NestJS	Framework Node.js theo kiến trúc module, controller, service, gateway.
TypeScript	Tăng an toàn kiểu dữ liệu và khả năng bảo trì mã nguồn.
Socket.IO	Giao tiếp realtime, quản lý room, broadcast, reconnect.

Công nghệ	Vai trò
JWT Authentication	Xác thực người dùng và bảo vệ API tài liệu.
MongoDB + Mongoose	Lưu User, Document và ánh xạ schema dữ liệu.
WebSocket Gateway	Nhận operation, xử lý OT, broadcast thay đổi.

4.3. Lý do chọn Socket.IO

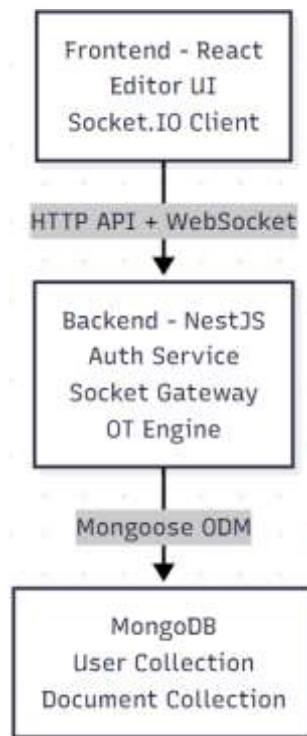
Socket.IO được chọn vì phù hợp với đề án realtime client-server: hỗ trợ room management sẵn, tự động reconnect, fallback transport khi WebSocket không khả dụng, tích hợp tốt với NestJS và React. So với WebSocket thuần, Socket.IO giúp giảm lượng mã phải tự viết cho quản lý room, reconnect và broadcast.

Giải pháp	Ưu điểm	Nhược điểm
Socket.IO	Room, reconnect, fallback, dễ dùng với NestJS.	Có overhead nhẹ so với WebSocket thuần.
WebSocket thuần	Chuẩn trình duyệt, nhẹ, latency thấp.	Phải tự xây dựng reconnect, room, heartbeat.
WebRTC	P2P, tiết kiệm server bandwidth.	Phức tạp, NAT traversal, khó đồng bộ và scale.
HTTP polling	Dễ triển khai.	Độ trễ cao, tốn request, không tối ưu realtime.

5. KIẾN TRÚC HỆ THỐNG

5.1. Mô hình tổng thể

Hệ thống sử dụng kiến trúc client-server tập trung. Server là authoritative source cho trạng thái tài liệu đang chỉnh sửa; các client giữ bản sao tạm thời để hiển thị và thao tác nhanh. Database MongoDB là nơi lưu trữ bền vững sau khi server auto-save hoặc khi người dùng lưu tài liệu.



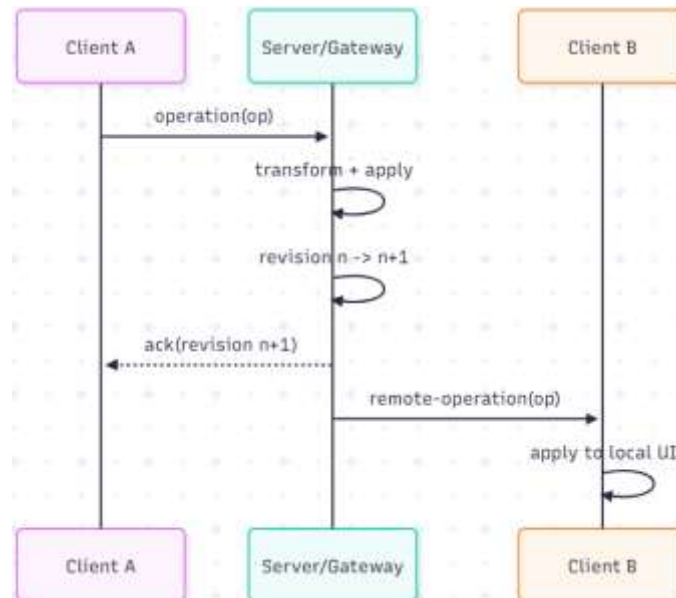
Hình 1. Kiến trúc tổng thể client-server của hệ thống

5.2. Vai trò các thành phần

Thành phần	Vai trò chính	Trạng thái quản lý
Client/Frontend	Hiển thị giao diện, nhận thao tác người dùng, gửi operation và render remote changes.	Local content, local revision, token, thông tin user.
Backend/Gateway	Điều phối realtime, kiểm tra revision, transform operation, broadcast đến room.	Authoritative in-memory document state, operation history, socket rooms.
Document Service	CRUD tài liệu, lưu/cập nhật dữ liệu vào MongoDB.	Document metadata và content.
Auth Service	Đăng ký, đăng nhập, phát hành và kiểm tra JWT.	User account, password hash, token payload.
MongoDB	Lưu dữ liệu bền vững.	User, Document, timestamps, collaborators.

5.3. Luồng hoạt động chính

1. Người dùng đăng ký/đăng nhập và nhận JWT token.
2. Frontend lưu token, dùng token cho các request HTTP cần xác thực.
3. Khi người dùng mở tài liệu, frontend kết nối Socket.IO và emit join-document với documentId.
4. Server đưa socket vào room tương ứng, gửi document-init gồm content, title và revision hiện tại.
5. Khi người dùng gõ hoặc xóa, frontend tạo operation và gửi đến server.
6. Server transform operation nếu revision của client đã cũ, áp dụng vào nội dung, tăng revision và lưu vào history.
7. Server gửi ack cho client gửi operation và broadcast remote-operation cho các client khác trong cùng room.
8. Sau một khoảng debounce, server auto-save trạng thái mới nhất vào MongoDB.



Hình 2. Sequence diagram rút gọn khi một user chỉnh sửa tài liệu

5.4. Vì sao hệ thống là hệ thống phân tán

Đặc trưng hệ thống phân tán	Biểu hiện trong đồ án
Nhiều node độc lập	Nhiều trình duyệt/client có local state riêng.
Giao tiếp qua mạng	HTTP API và WebSocket/Socket.IO.
Không chia sẻ bộ nhớ	Client không truy cập trực tiếp memory của nhau; mọi thay đổi đi qua server.
Xử lý đồng thời	Nhiều user có thể gửi operation cùng lúc.

Đặc trưng hệ thống phân tán	Biểu hiện trong đồ án
Lỗi cục bộ	Một client mất kết nối không làm các client khác dừng chỉnh sửa.
Đồng bộ trạng thái	OT + revision giúp các client hội tụ về một nội dung tài liệu.

6. THIẾT KẾ DỮ LIỆU VÀ LƯU TRỮ

6.1. User Collection

Field	Type	Description
_id	ObjectId	ID duy nhất của người dùng
email	String	Email đăng nhập của người dùng (unique)
password	String	Mật khẩu đã được mã hóa bằng bcryptjs
fullname	String	Họ và tên người dùng
clientId	String	Định danh client/socket của người dùng (tùy chọn)
createdAt	Date	Thời gian tạo tài khoản
updatedAt	Date	Thời gian cập nhật gần nhất

6.2. Document Collection

Field	Type	Description
_id	ObjectId	ID duy nhất của tài liệu
title	String	Tiêu đề tài liệu
content	String	Nội dung hiện tại của tài liệu
userId	ObjectId	ID chủ sở hữu tài liệu (reference User)
collaborators	[ObjectId]	Danh sách cộng tác viên

createdAt	Date	Thời gian tạo tài liệu
updatedAt	Date	Thời gian cập nhật gần nhất
lastModifiedBy	ObjectId	Người chỉnh sửa gần nhất
version	Number	Phiên bản hiện tại của tài liệu

6.3. Trạng thái tài liệu trong memory

Để xử lý realtime nhanh, backend giữ một cấu trúc DocumentState trong memory cho mỗi tài liệu đang có người chỉnh sửa. Khi room trống, server có thể lưu lần cuối vào MongoDB rồi giải phóng trạng thái trong memory để tránh tốn RAM.

Field	Type	Description
documentId	string	ID của tài liệu
title	string	Tiêu đề tài liệu
content	string	Nội dung hiện tại của tài liệu
revision	number	Phiên bản hiện tại dùng cho OT synchronization
history	Operation[]	Danh sách các operation đã thực hiện

6.4. Lưu tài liệu

Hệ thống sử dụng chiến lược full document save ở mức hiện tại: server lưu toàn bộ title và content hiện tại của tài liệu vào MongoDB. Để giảm tần suất ghi database, cơ chế auto-save dùng debounce khoảng 3 giây: nếu trong 3 giây có thêm chỉnh sửa mới, timer cũ bị hủy và timer mới được đặt lại. Nhờ vậy, nhiều thao tác gõ liên tiếp chỉ tạo một lần lưu cuối cùng.

User edit -> scheduleAutoSave(documentId)

Nếu có edit mới trong 3 giây:

hủy timer cũ

đặt timer mới

Sau 3 giây không có edit:

update MongoDB với content/title mới nhất

Ưu điểm của full save là đơn giản, dễ khôi phục và phù hợp với tài liệu văn bản nhỏ/trung bình. Nhược điểm là khi tài liệu rất lớn, mỗi lần lưu sẽ ghi nhiều dữ liệu hơn so với incremental save. Hướng phát triển là lưu thêm operation log hoặc snapshot để hỗ trợ version history và phục hồi theo revision.

7. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU

7.1. Tổng quan Operational Transformation

Operational Transformation là cơ chế biến đổi thao tác chỉnh sửa khi có nhiều thao tác đồng thời. Thay vì khóa tài liệu để chỉ một người được sửa, hệ thống cho phép nhiều client gửi thao tác độc lập. Server nhận operation, kiểm tra revision, transform vị trí nếu cần, rồi áp dụng lên bản tài liệu chính thức.

Mỗi thay đổi cơ bản được mô hình hóa thành operation. Trong sản phẩm hiện tại, hai loại operation quan trọng là insert và delete. Operation luôn đi kèm documentId, clientId và revision tại thời điểm client tạo thao tác.

Insert operation:

```
{
  type: 'insert',
  position: 5,
  char: 'a',
  clientId: 'socket-id',
  documentId: 'doc-id',
  revision: 3
}
```

Delete operation:

```
{
  type: 'delete',
  position: 10,
  length: 2,
  clientId: 'socket-id',
  documentId: 'doc-id',
  revision: 3
}
```

7.2. Revision system

Revision là bộ đếm phiên bản logic của tài liệu. Ban đầu revision có thể là 0. Mỗi operation hợp lệ sau khi được server áp dụng sẽ làm revision tăng lên 1. Client gửi revision hiện tại của mình để server biết client đang dựa trên trạng thái nào.

Document: "Hello", revision = 0
A insert 'X' at position 0 -> "XHello", revision = 1
B delete at position 2 -> "XHllo", revision = 2
C insert 'Y' at position 5 -> "XHlloY", revision = 3

Nếu client gửi operation với revision nhỏ hơn server revision, nghĩa là client đang chậm hơn. Server sẽ lấy các operation trong history từ revision của client đến revision hiện tại, rồi transform operation mới qua từng operation đã áp dụng.

7.3. Các quy tắc transform cơ bản

Trường hợp	Quy tắc xử lý	Ví dụ
Insert - Insert	Nếu một insert trước đó nằm trước hoặc cùng vị trí, insert đến sau có thể phải dịch sang phải. Khi cùng vị trí dùng tie-breaking theo clientId để deterministic.	A insert X tại 2; B insert Y tại 2 -> B có thể dịch sang 3.
Insert - Delete	Nếu insert đã áp dụng nằm trước hoặc tại vị trí delete của operation đến sau, vị trí delete dịch sang phải.	A insert X tại 2; B delete tại 2 -> B delete tại 3.
Delete - Insert	Nếu delete đã áp dụng nằm trước vị trí insert, vị trí insert phải dịch sang trái theo số ký tự đã xóa, không vượt quá điểm bắt đầu delete.	A delete pos 1 len 2; B insert tại 3 -> B dịch về 1.
Delete - Delete	Nếu hai vùng xóa chồng lấn, operation đến sau cần giảm length hoặc trở thành no-op với phần đã bị xóa.	A xóa "bc"; B xóa "c" -> B thành no-op.

7.4. Ví dụ xử lý xung đột

Tài liệu ban đầu: "Hello", revision = 0

Client A: insert('X', position = 2), revision = 0

Client B: insert('Y', position = 2), revision = 0

Server nhận A trước:

"Hello" -> "HeXllo", revision = 1

Server nhận B sau:

B đang dựa trên revision 0, server đã ở revision 1

Transform B qua operation A:

position 2 -> 3

Apply B:

"HeXllo" -> "HeXYllo", revision = 2

Kết quả: cả X và Y đều được giữ lại, mọi client cuối cùng nhận cùng nội dung.

7.5. Consistency model

Hệ thống sử dụng mô hình eventual consistency. Trong một khoảng thời gian rất ngắn do độ trễ mạng, hai client có thể hiển thị trạng thái khác nhau. Tuy nhiên, sau khi server xử lý và broadcast các operation, tất cả client sẽ hội tụ về cùng một nội dung tài liệu và cùng revision mới nhất.

Tiêu chí	Strong consistency	Eventual consistency trong hệ thống
Cảm nhận người dùng	Có thể chậm do phải chờ khóa/xác nhận tuyệt đối.	Mượt hơn vì client có thể thao tác gần realtime.
Xử lý xung đột	Thường dùng lock hoặc transaction mạnh.	Dùng OT và revision.
Phù hợp đồ án	Không tối ưu cho realtime editor đơn giản.	Phù hợp vì cần tốc độ và hội tụ cuối cùng.

8. GIAO THỨC TRAO ĐỔI DỮ LIỆU

8.1. HTTP REST API

HTTP API được dùng cho các tác vụ không yêu cầu realtime liên tục như đăng ký, đăng nhập và quản lý tài liệu. Các API tài liệu yêu cầu header Authorization dạng Bearer token.

Nhóm API	Endpoint	Mục đích	Xác thực
Auth	POST /auth/register	Tạo tài khoản mới.	Không
Auth	POST /auth/login	Đăng nhập và nhận access_token.	Không
Document	GET /documents	Lấy danh sách tài liệu của người dùng.	Bearer JWT
Document	POST /documents	Tạo tài liệu mới.	Bearer JWT
Document	GET /documents/:id	Lấy chi tiết tài liệu.	Bearer JWT
Document	PATCH /documents/:id	Cập nhật title/content.	Bearer JWT
Document	DELETE /documents/:id	Xóa tài liệu.	Bearer JWT

POST /auth/login

Content-Type: application/json

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "user": {
    "id": "507f1f77bcf86cd799439011",
    "email": "user@example.com",
    "fullname": "John Doe"
  }
}
```

8.2. WebSocket/Socket.IO Events

WebSocket/Socket.IO được dùng cho luồng chỉnh sửa realtime. Mỗi tài liệu tương ứng một room; client mở tài liệu nào thì join vào room của tài liệu đó.

Chiều	Event	Chức năng
Client -> Server	join-document	Client tham gia room chỉnh sửa tài liệu, server gửi document-init.
Client -> Server	operation	Gửi thao tác insert/delete kèm revision.
Client -> Server	save-document	Lưu tài liệu thủ công.
Client -> Server	title-change	Cập nhật tiêu đề realtime.
Client -> Server	user_start_typing/user_stop_typing	Thông báo trạng thái đang gõ.
Server -> Client	document-init	Gửi content, title, revision khi client join.
Server -> Client	remote-operation	Broadcast operation từ user khác.
Server -> Client	ack	Xác nhận operation đã xử lý.
Server -> Client	auto-saved	Thông báo tài liệu đã được lưu tự động.
Server -> Client	active-users	Gửi số lượng user/socket trong room.
Server -> Client	user-joined/user-left	Thông báo có user vào/rời tài liệu.
Server -> Client	typing_indicator	Hiển thị người dùng đang gõ.

```
socket.emit('join-document', {  
  documentId: '507f1f77bcf86cd799439012',  
  fullname: 'John Doe',  
  email: 'john@example.com'  
});
```

```
socket.emit('operation', {
```

```
type: 'insert',
position: 5,
char: 'a',
clientId: socket.id,
documentId: 'doc-id',
revision: 3
});
```

8.3. Luồng reconnect

Khi client mất mạng hoặc trình duyệt tạm thời mất kết nối, Socket.IO tự động thử reconnect. Sau khi reconnect thành công, frontend cần join lại document room và nhận document-init mới nhất từ server. Cách này đảm bảo client đồng bộ lại với trạng thái chính thức thay vì tiếp tục dựa trên dữ liệu cũ.

Mất kết nối -> Socket.IO reconnect -> xác thực token -> join-document
-> server gửi content/revision mới nhất -> client cập nhật local state -> tiếp tục chỉnh sửa

9. QUẢN LÝ NGƯỜI DÙNG ĐANG CHỈNH SỬA

Mỗi tài liệu được ánh xạ thành một Socket.IO room. Khi user mở tài liệu, socket của user join vào room có tên là documentId. Vì vậy, danh sách socket trong room phản ánh các người dùng/phiên kết nối đang chỉnh sửa tài liệu đó.

Room "doc-1": socketA, socketB, socketC

Room "doc-2": socketD

active-users(doc-1) = 3

active-users(doc-2) = 1

9.1. Presence và typing indicator

- user-joined: thông báo khi có socket mới tham gia document room.
- user-left: thông báo khi socket rời room hoặc disconnect.
- active-users: cập nhật số lượng người đang online trong tài liệu.
- typing_indicator: cho biết một người dùng đang bắt đầu hoặc dừng gõ.

9.2. Khả năng mở rộng con trỏ cộng tác

Đề bài khuyến khích hiển thị vị trí con trỏ hoặc vùng chỉnh sửa. Trong sản phẩm hiện tại, README xem collaborative cursor là hướng phát triển. Có thể bổ sung bằng cách client gửi cursor-move gồm position/selection, server broadcast remote-cursor đến các user khác trong cùng room. Để tránh quá tải mạng, nên debounce khoảng 100-200ms cho event cursor.

```
socket.emit('cursor-move', {  
  documentId: 'doc-id',  
  position: 25,  
  selectionStart: 25,  
  selectionEnd: 30,  
  fullname: 'John Doe'  
});
```

10. XỬ LÝ ĐỒNG THỜI, XUNG ĐỘT VÀ LỖI

10.1. Xử lý nhiều client đồng thời

Backend Node.js/NestJS xử lý nhiều kết nối socket bằng mô hình event-driven. Mỗi operation được server xử lý tuần tự theo thứ tự nhận tại gateway. Điều này giúp server gán revision tăng đơn điệu, tránh hai operation cùng được áp dụng với cùng một revision.

10.2. Xử lý client gửi revision cũ

Server currentRevision = 10

Client gửi operation với revision = 7

Server lấy history các operation đã áp dụng từ revision 7 đến 10:

history[8], history[9], history[10]

Transform operation mới qua từng operation trong history

Apply operation đã transform

Tăng revision lên 11

Broadcast operation đã transform

10.3. Xử lý lỗi thường gặp

Lỗi	Nguyên nhân	Cách xử lý
-----	-------------	------------

Lỗi	Nguyên nhân	Cách xử lý
EADDRINUSE	Port backend/frontend đã có process khác dùng.	Đổi PORT trong .env hoặc kill process cũ.
MongoDB connection failed	MongoDB chưa chạy hoặc URI sai.	Khởi động mongod, kiểm tra MONGODB_URI.
Frontend không kết nối backend	Backend chưa chạy, CORS/URL sai.	Kiểm tra http://localhost:3001, CLIENT_URL, port.
WebSocket connection failed	Sai port, CORS, firewall hoặc server gateway lỗi.	Kiểm tra cấu hình Socket.IO và network.
Token expired	JWT hết hạn.	Đăng nhập lại hoặc điều chỉnh JWT_EXPIRES_IN.

11. BẢO MẬT HỆ THỐNG

11.1. Authentication và Authorization

Hệ thống dùng JWT để xác thực. Sau khi đăng nhập thành công, server trả access_token cho client. Client gửi token trong header Authorization: Bearer <token> khi gọi các API quản lý tài liệu. Server verify chữ ký JWT bằng JWT_SECRET trong file .env.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

11.2. Bảo vệ mật khẩu

Mật khẩu không được lưu dạng plaintext. Khi đăng ký, server hash mật khẩu bằng bcryptjs rồi lưu hash vào MongoDB. Khi đăng nhập, server dùng bcrypt.compare để so sánh mật khẩu người dùng nhập với hash đã lưu.

11.3. CORS và cấu hình môi trường

Các thông tin nhạy cảm như MONGODB_URI, JWT_SECRET, CLIENT_URL được đặt trong file .env thay vì hard-code trong source code. Trong môi trường production, CORS nên giới hạn origin về domain frontend thật thay vì dùng wildcard.

```
PORT=3001
MONGODB_URI=mongodb://127.0.0.1:27017/collaborative-editor
```

```
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:3000
```

11.4. Input validation và chống lạm dụng

- Dùng DTO validation cho email, password, title và content để tránh dữ liệu sai định dạng.
- Không render nội dung người dùng bằng dangerouslySetInnerHTML nếu chưa sanitize để giảm nguy cơ XSS.
- Có thể bổ sung rate limiting cho HTTP API và WebSocket operation để tránh spam/DoS.
- Kiểm tra quyền truy cập tài liệu dựa trên owner/collaborators trước khi cho phép đọc hoặc chỉnh sửa.

12. THỬ NGHIỆM VÀ ĐÁNH GIÁ

12.1. Mục tiêu kiểm thử

- Kiểm tra đăng ký, đăng nhập và bảo vệ API bằng JWT.
- Kiểm tra CRUD tài liệu: tạo, xem danh sách, xem chi tiết, cập nhật, xóa.
- Kiểm tra realtime: nhiều client cùng join một document room và nhận thay đổi của nhau.
- Kiểm tra OT khi nhiều client chỉnh sửa cùng vị trí hoặc thao tác insert/delete chồng lấn.
- Kiểm tra reconnect và khả năng đồng bộ lại trạng thái mới nhất.
- Đánh giá latency, memory và CPU trong các tình huống nhiều user.

12.2. Kiểm thử API bằng cURL/Postman

```
# Register
curl -X POST http://localhost:3001/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123","fullname":"Test User"}'

# Login
curl -X POST http://localhost:3001/auth/login \
```

```
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","password":"password123"}'
```

Create document

```
curl -X POST http://localhost:3001/documents \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "Content-Type: application/json" \
-d '{"title":"Test Doc","content":"Test content"}'
```

12.3. Kiểm thử realtime

Test case	Cách thực hiện	Kết quả mong đợi
2 user cùng mở một tài liệu	Mở hai trình duyệt/tài khoản, join cùng documentId.	Cả hai nhận document-init và active-users = 2.
User A gõ nội dung	A insert ký tự trong editor.	B nhận remote-operation và UI cập nhật gần realtime.
Hai user insert cùng vị trí	A và B cùng insert ở position 0 với cùng revision.	Server transform deterministic, cả hai ký tự đều được giữ.
Delete/insert chồng lấn	A xóa một đoạn, B insert trong/near đoạn đó.	Operation được transform, không corrupt content.
Reconnect	Ngắt mạng client rồi kết nối lại.	Client join lại room, nhận content/revision mới nhất.
Auto-save	Chỉnh sửa rồi dừng gõ trên 3 giây.	Server lưu document vào MongoDB và emit auto-saved.

12.4. Kết quả đánh giá hiệu năng mẫu

Theo README, môi trường thử nghiệm mẫu gồm backend NestJS/Node.js v18, MongoDB local, mạng LAN độ trễ dưới 5ms và máy 8GB RAM. Bảng sau tóm tắt các chỉ số được dùng để đánh giá cảm nhận realtime khi số lượng người dùng tăng.

Chỉ số	2 users	5 users	10 users
Insert latency	15ms	25ms	45ms
Transform	2ms	3ms	5ms

Chỉ số	2 users	5 users	10 users
latency			
Broadcast latency	10ms	12ms	20ms
Total RTT	27ms	40ms	70ms
Memory server	50MB	80MB	120MB
CPU usage	5%	8%	15%

12.5. Đánh giá kết quả

- Với 2-5 users, độ trễ dưới 50ms cho cảm nhận gần realtime.
- Ở 10 users, RTT khoảng 70ms vẫn chấp nhận được cho đồ án và môi trường LAN.
- OT transform có chi phí thấp vì operation nhỏ và history được giới hạn.
- Broadcast chỉ gửi đến room của tài liệu, giúp giảm overhead so với broadcast toàn hệ thống.
- Auto-save debounce giúp giảm số lần ghi MongoDB khi người dùng gõ liên tục.
- Hạn chế chính là kiến trúc single server; nếu server dừng, realtime editing tạm dừng cho đến khi server khởi động lại.

13. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

13.1. Hạn chế hiện tại

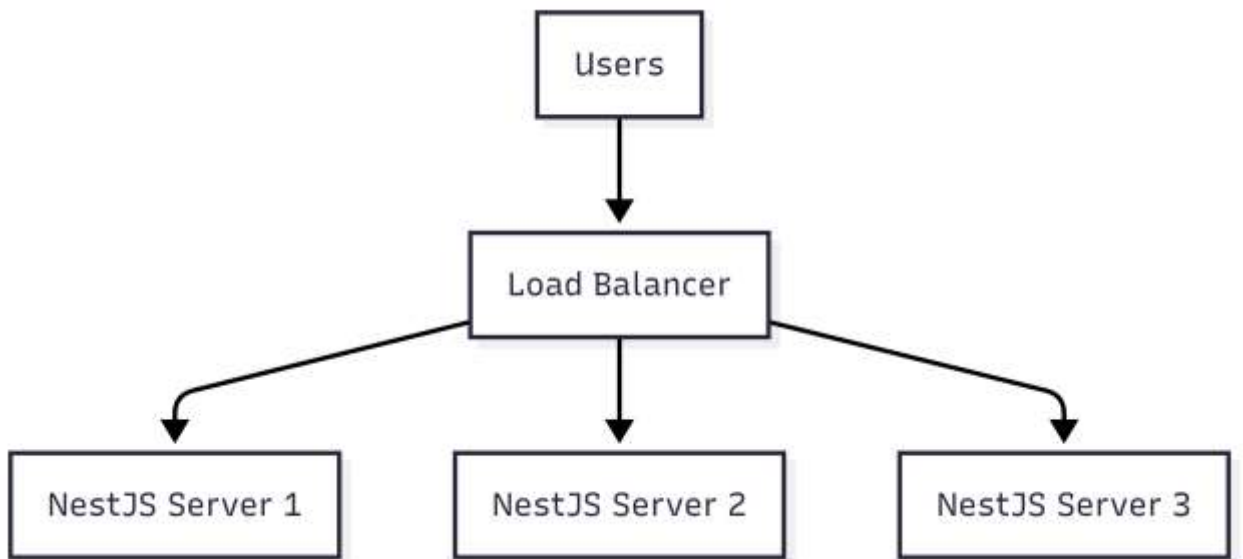
- Hệ thống chủ yếu hỗ trợ plain text, chưa có rich text formatting như heading, bold, italic, image, table.
- Operation history hiện dùng cho transform trong memory, chưa phải lịch sử chỉnh sửa bền vững để người dùng xem lại.
- Chưa có role-based permission đầy đủ như owner, editor, viewer, commenter.
- Chưa triển khai collaborative cursor/selection hoàn chỉnh.
- Chưa hỗ trợ offline editing đầy đủ; khi reconnect chủ yếu đồng bộ lại từ server.
- Single NestJS server là single point of failure và giới hạn khả năng scale ngang.

13.2. Hướng phát triển

Hướng phát triển	Mô tả triển khai đề xuất
Version history/snapshots	Lưu snapshot theo revision và operation log để khôi phục tài liệu tại thời điểm bất kỳ.

Hướng phát triển	Mô tả triển khai đề xuất
Undo/redo cộng tác	Dùng inverse operation và transform inverse operation như thao tác bình thường.
Collaborative cursor	Thêm cursor-move/remote-cursor, debounce 100-200ms, hiển thị màu riêng từng user.
Offline editing	Queue operations trong LocalStorage/IndexedDB khi mất mạng, sync và transform khi kết nối lại.
Permission system	Bổ sung role owner/editor/viewer/commenter, kiểm tra quyền ở REST API và gateway.
Multi-server scaling	Dùng Socket.IO Redis Adapter, Redis Pub/Sub, load balancer và sticky session.
Rich text editor	Tích hợp Quill/TipTap; cân nhắc OT cho rich text hoặc CRDT như Yjs.
Export PDF/DOCX	Cho phép người dùng xuất tài liệu ra định dạng nộp/báo cáo.

Multi-server đề xuất:



Các server đồng bộ broadcast qua Redis Pub/Sub.
MongoDB Atlas/Replica Set lưu dữ liệu bền vững.

14. KẾT LUẬN

Đồ án Collaborative Document Editing System đã đáp ứng các yêu cầu chính của đề bài về một hệ thống chỉnh sửa văn bản cộng tác theo thời gian thực. Hệ thống

cho phép người dùng đăng ký, đăng nhập, tạo và chỉnh sửa tài liệu; nhiều client có thể cùng tham gia một tài liệu; thay đổi được truyền qua WebSocket/Socket.IO; server xử lý xung đột bằng Operational Transformation và lưu dữ liệu vào MongoDB.

Về mặt hệ thống phân tán, đồ án thể hiện rõ các vấn đề quan trọng: giao tiếp mạng, xử lý đồng thời, trạng thái phân tán giữa nhiều client, đồng bộ dữ liệu, lỗi kết nối và nhất quán cuối cùng. Việc sử dụng revision và operation history giúp server xác định client nào đang chậm và transform thao tác để tránh mất dữ liệu.

Trong phạm vi đồ án sinh viên, kiến trúc client-server tập trung là lựa chọn hợp lý vì dễ triển khai, dễ kiểm soát trạng thái và phù hợp với Socket.IO/NestJS. Các hướng mở rộng như version history, offline editing, collaborative cursor, permission system và multi-server scaling có thể giúp hệ thống tiến gần hơn tới một sản phẩm thực tế.

TÀI LIỆU THAM KHẢO

- [1] NestJS Documentation – <https://docs.nestjs.com>
- [2] Socket.IO Documentation – <https://socket.io/docs/v4>
- [3] React Documentation – <https://react.dev>
- [4] MongoDB Documentation – <https://www.mongodb.com/docs>
- [5] Ellis, C. A., Gibbs, S. J. (1989), *Concurrency Control in Groupware Systems*, ACM SIGMOD International Conference on Management of Data.
- [6] Sun, C., *Operational Transformation FAQ* – <https://www3.ntu.edu.sg/scse/staff/czsun/projects/otfaq>
- [7] Redis Documentation – <https://redis.io/docs>
- [8] Fowler, M. (2002), *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional.
- [9] Tanenbaum, A. S., Steen, M. V. (2017), *Distributed Systems: Principles and Paradigms*, Pearson Education.
- [10] Coulouris, G., Dollimore, J., Kindberg, T., Blair, G. (2011), *Distributed Systems: Concepts and Design*, Addison-Wesley.